

Rajarata University of Sri Lanka
Department of Computing



ICT1402

PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 08

Program Control Structures – Decision Making & Branching

K.A.S.H. Kulathilake

Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

- At the end of this lecture students should be able to;
 - Define the operation of if, if-else, nested if-else, switch and conditional operator.
 - Justify the control flow of the program under the aforementioned C language constructs.
 - Apply taught concepts for writing programs.

Decision Making with C

- The C programming language also provides several decision-making constructs, which are:
 - The if statement
 - The switch statement
 - The conditional operator

if Statement

- The general format of the 'if' statement is as follows:

```
if ( expression )  
    Program statement
```

or

```
if ( expression )  
{  
    block of statements;  
}
```

- Similarly, in the program statement

```
if ( count > COUNT_LIMIT )  
    printf ("Count limit exceeded\n");
```

- The printf statement is executed *only* if the value of count is greater than the value of COUNT_LIMIT; otherwise, it is ignored.

if Statement (Cont...)

```
int main (void)
{
    int number;
    printf ("Type in your number: ");
    scanf ("%i", &number);
    if ( number < 0 )
    {
        number = -number;
    }
    printf ("The absolute value is %i\n", number);
    return 0;
}
```

If entered number is < 0 the controller passes in to the if block and negate the value of number. Then continue with the printf statement.

If entered number is ≥ 0 the controller ignores the if block and directly passes to printf statement.

It is no need to specify the scope using `{}` of the if block if it has single statement.

if-else Construct

- The general format of the 'if-else' statement is as follows;

```
if (expression)
{
    program statement 1;
}
else
{
    program statement 2;
}
```

- The if-else is actually just an extension of the general format of the if statement.
- If the result of the evaluation of *expression* is TRUE, *program statement 1*, which immediately follows, is executed; otherwise, *program statement 2* is executed.

if-else Construct (Cont...)

- Similarly, in the program statement:

```
if ( count > COUNT_LIMIT )  
    printf ("Count limit exceeded\n");  
else  
    printf ("Count limit is not exceeded\n");
```

- **Count limit exceeded** message is printed only if the value of count is greater than the value of COUNT_LIMIT; otherwise, it executes the statement within the else block which is **Count limit is not exceeded**.

if-else Construct (Cont...)

```
#include <stdio.h>
int main (void)
{
    int number_to_test, remainder;
    printf ("Enter your number to be tested.: ");
    scanf ("%i", &number_to_test);
    remainder = number_to_test % 2;
    if ( remainder == 0 )
        printf ("The number is even.\n");
    if ( remainder != 0 )
        printf ("The number is odd.\n");
    return 0;
}
```


Compound Relational Test

- A *compound relational test* is simply one or more simple relational tests joined by either the *logical AND* or the *logical OR* operator.
- These operators are represented by the character pairs `&&` and `||`, respectively.
- As an example, the C statement

```
if ( grade >= 70 && grade <= 79 )  
    ++grades_70_to_79;
```
- increments the value of `grades_70_to_79` only if the value of `grade` is greater than or equal to 70 *and* less than or equal to 79.
- In the same way, the statement

```
if ( index < 0 || index > 99 )  
    printf ("Error - index out of range\n");
```
- causes execution of the `printf` statement if `index` is less than 0 *or* greater than 99.

Compound Relational Test (Cont...)

- The compound operators can be used to form extremely complex expressions in C.
- The C language grants the programmer ultimate flexibility in forming expressions.
- This flexibility is a capability that is often abused.
- Simpler expressions are almost always easier to read and debug.

Compound Relational Test (Cont...)

- When forming compound relational expressions, liberally use parentheses to aid readability of the expression and to avoid getting into trouble because of a mistaken assumption about the precedence of the operators in the expression.
- You can also use blank spaces to aid in the expression's readability.
- An extra blank space around the `&&` and `||` operators visually sets these operators apart from the expressions that are being joined by these operators.

Compound Relational Test (Cont...)

- Candidate Selection ?

```
#include <stdio.h>
int main (void)
{
    int appointmentNo, age, score;

    printf ("Enter the appointment number : ");
    scanf ("%i", &appointmentNo);
    printf ("Enter age : ");
    scanf ("%i", &age);
    printf ("Enter score : ");
    scanf ("%i", &score);

    if ( (appointmentNo <= 30 && age >= 18) || score >= 40 )
        printf ("Ticket issued.\n");
    else
        printf ("You are not eligible.\n");
    return 0;
}
```

Nested if Statement

- In the general format of the if statement, remember that if the result of evaluating the expression inside the parentheses is TRUE, the statement that immediately follows is executed.
- It is perfectly valid that this program statement be another if statement, as in the following statement:

```
if ( score >= 40 )  
    if ( age >= 18 )  
        printf ("Issue Ticket\n");
```

- If the value of score is ≥ 40 , the following statement is executed, which is another if statement.
- This if statement compares the value of age ≥ 18 .
- If the two values are equal, the message “Issue Ticket” is displayed at the terminal.

Nested if Statement (Cont...)

- What happen if we add an else clause

```
if ( score >= 40 )  
    if ( age >= 18 )  
        printf ("Issue Ticket\n");  
    else  
        printf ("Not in age\n");
```

- In this example else clause belongs to the closest if statement.

Nested if Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    int age,score;

    printf ("Enter age : ");
    scanf ("%i", &age);
    printf ("Enter score : ");
    scanf ("%i", &score);

    if ( score >= 40 )
        if ( age >= 18 )
            printf ("Issue Ticket\n");
        else
            printf ("Not in age\n");
    return 0;
}
```

What happen when you
enter following details?
Score = 40 and age 18
Score =20 and age 18
Score = 40 and age 10

Nested if Statement (Cont...)

- Another approach:

```
if ( score >= 40 )  
{  
    if ( age >= 18 )  
    {  
        printf ("Issue Ticket\n");  
    }  
}  
else  
{  
    printf ("Not in age\n");  
}
```

- In this example else clause belongs to the outer if statement.

Nested if Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    int age,score;

    printf ("Enter age : ");
    scanf ("%i", &age);
    printf ("Enter score : ");
    scanf ("%i", &score);

    if ( score >= 40 )
    {
        if ( age >= 18 )
        {
            printf ("Issue Ticket\n");
        }
    }
    else
    {
        printf ("Not in age\n");
    }
    return 0;
}
```

What happen when you
enter following details?
Score = 40 and age 18
Score =20 and age 18
Score = 40 and age 10

Nested if Statement (Cont...)

- Complete version

```
#include <stdio.h>
int main (void)
{
    int age,score;

    printf ("Enter age : ");
    scanf ("%i", &age);
    printf ("Enter score : ");
    scanf ("%i", &score);

    if ( score >= 40 )
        if ( age >= 18 )
            printf ("Issue Ticket\n");
        else
            printf ("Not in age\n");
    else
        printf("Not scored\n");
    return 0;
}
```

What happen when you enter following details?

Score = 40 and age 18

Score =20 and age 18

Score = 40 and age 10

The else-if Statement

Suppose you want to make three different decisions based on the value of an input number. E.g. if the input number < 0 , you execute program statement 1, if the input number $== 0$, you execute program statement 2 and if the input number > 0 , you execute program statement 3.

- Without if- else-if approach

```
if ( expression 1 )  
    program statement 1  
else  
    if ( expression 2 )  
        program statement 2  
    else  
        program statement 3
```

```
if (input < 0 )  
    print ("-");  
else  
    if ( input == 0 )  
        print ("0");  
    else  
        print ("+" );
```

The else-if Statement (Cont...)

- With if- else-if approach

```
if ( expression 1 )  
    program statement 1  
else if ( expression 2 )  
    program statement 2  
else  
    program statement 3
```

```
if (input < 0 )  
    print ("-");  
else if ( input == 0 )  
    print ("0");  
else  
    print ("+");
```

This method of formatting improves the readability of the statement and makes it clearer that a three-way decision is being made.

The else-if Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    char c;
    printf ("Enter a single character:\n");
    scanf ("%c", &c);
    if ( (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'))
        printf ("It's an alphabetic character.\n");
    else if ( c >= '0' && c <= '9' )
        printf ("It's a digit.\n");
    else
        printf ("It's a special character.\n");
    return 0;
}
```

The else-if Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    float value1, value2;
    char operator;
    printf ("Type in your expression.\n");
    scanf ("%f %c %f", &value1, &operator, &value2);
    if ( operator == '+' )
        printf (".2f\n", value1 + value2);
    else if ( operator == '-' )
        printf (".2f\n", value1 - value2);
    else if ( operator == '*' )
        printf (".2f\n", value1 * value2);
    else if ( operator == '/' )
        printf (".2f\n", value1 / value2);
    return 0;
}
```

The else-if Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    float value1, value2;
    char operator;
    printf ("Type in your expression.\n");
    scanf ("%f %c %f", &value1, &operator, &value2);
    if ( operator == '+' )
        printf (".2f\n", value1 + value2);
    else if ( operator == '-' )
        printf (".2f\n", value1 - value2);
    else if ( operator == '*' )
        printf (".2f\n", value1 * value2);
    else if ( operator == '/' )
        if ( value2 == 0 )
            printf ("Division by zero.\n");
        else
            printf (".2f\n", value1 / value2);
    else
        printf ("Unknown operator.\n");
    return 0;
}
```

switch Statement

- Within the type of if-else statement chain the value of a variable is successively compared against different values.
- It is so commonly used when developing programs that a special program statement exists in the C language for performing precisely this function.
- The name of the statement is the switch statement.

switch Statement (Cont...)

- The general format of switch statement is as follows;

```
switch ( expression )
{
    case value1:
        program statement
        program statement
        ...
        break;
    case value2:
        program statement
        program statement
        ...
        break;
    ...
    case valuen:
        program statement
        program statement
        ...
        break;
    default:
        program statement
        program statement
        ...
        break;
}
```

switch Statement (Cont...)

- The *expression* enclosed within parentheses is successively compared against the values $value_1$, $value_2$, ..., $value_n$, which must be simple constants or constant expressions.
- If a case is found whose value is equal to the value of *expression*, the program statements that follow the case are executed.
- Note that when more than one such program statement is included, they do *not* have to be enclosed within braces.

switch Statement (Cont...)

- The break statement signals the end of a particular case and causes execution of the switch statement to be terminated.
- Remember to include the break statement at the end of every case.
- Forgetting to do so for a particular case causes program execution to continue into the next case whenever that case gets executed.

switch Statement (Cont...)

- The special optional case called default is executed if the value of *expression* does not match any of the case values.
- This is conceptually equivalent to the “fall through” else that you used in the previous if-else-if example.

switch Statement (Cont...)

```
#include <stdio.h>
int main (void)
{
    float value1, value2;
    char operator;
    printf ("Type in your expression.\n");
    scanf ("%f %c %f", &value1, &operator, &value2);
    switch (operator)
    {
        case '+':
            printf (".2f\n", value1 + value2);
            break;
        case '-':
            printf (".2f\n", value1 - value2);
            break;
```

switch Statement (Cont...)

```
case '*':
    printf ( "%.2f\n", value1 * value2 );
break;
case '/':
    if ( value2 == 0 )
        printf ( "Division by zero.\n" );
    else
        printf ( "%.2f\n", value1 / value2 );
break;
default:
    printf ( "Unknown operator.\n" );
break;
}
return 0;
}
```

switch Statement (Cont...)

- It is a good programming habit to remember to include the break at the end of every case.
- When writing a switch statement, bear in mind that no two case values can be the same.
- However, you can associate more than one case value with a particular set of program statements.
- This is done simply by listing the multiple case values (with the keyword case before the value and the colon after the value in each case) before the common statements that are to be executed.

switch Statement (Cont...)

```
switch (operator)
{
    ...
    case '*':
    case 'x':
        printf ( "%.2f\n", value1 * value2);
        break;
    ...
}
```


The Conditional Operator

- The conditional operator is a *ternary* operator; that is, it takes three operands.
- The two symbols that are used to denote this operator are the question mark (?) and the colon (:).
- The first operand is placed before the ?, the second between the ? and the :, and the third after the :.

The Conditional Operator (Cont...)

- The general format of the conditional operator is:

`condition ? expression1 : expression2`

- Where *condition* is an expression, usually a relational expression, that is evaluated first whenever the conditional operator is encountered.
- If the result of the evaluation of *condition* is TRUE (that is, nonzero), then *expression1* is evaluated and the result of the evaluation becomes the result of the operation.
- If *condition* evaluates FALSE (that is, zero), then *expression2* is evaluated and its result becomes the result of the operation.

The Conditional Operator (Cont...)

- The conditional operator is most often used to assign one of two values to a variable depending upon some condition.
- For example, suppose you have an integer variable x and another integer variable s .
- If you want to assign -1 to s if x were less than zero, and the value of x^2 to s otherwise, the following statement could be written:

$$s = (x < 0) ? -1 : x * x;$$

The Conditional Operator (Cont...)

- Examples
- State what happen in following statements;

```
maxValue = ( a > b ) ? a : b;
```

```
sign = ( number < 0 ) ? -1 : (( number == 0 ) ? 0 : 1);
```

```
printf ("Sign = %i\n",  
        ( number < 0 ) ? -1 : ( number == 0 ) ? 0 : 1);
```

Objective Re-cap

- Now you should be able to:
 - Define the operation of if, if-else, nested if-else, switch and conditional operator.
 - Justify the control flow of the program under the aforementioned C language constructs.
 - Apply taught concepts for writing programs.

References

- Chapter 06 - Programming in C, 3rd Edition, Stephen G. Kochan

Q & A

Next: Program Control Structures – Repetition and Loops